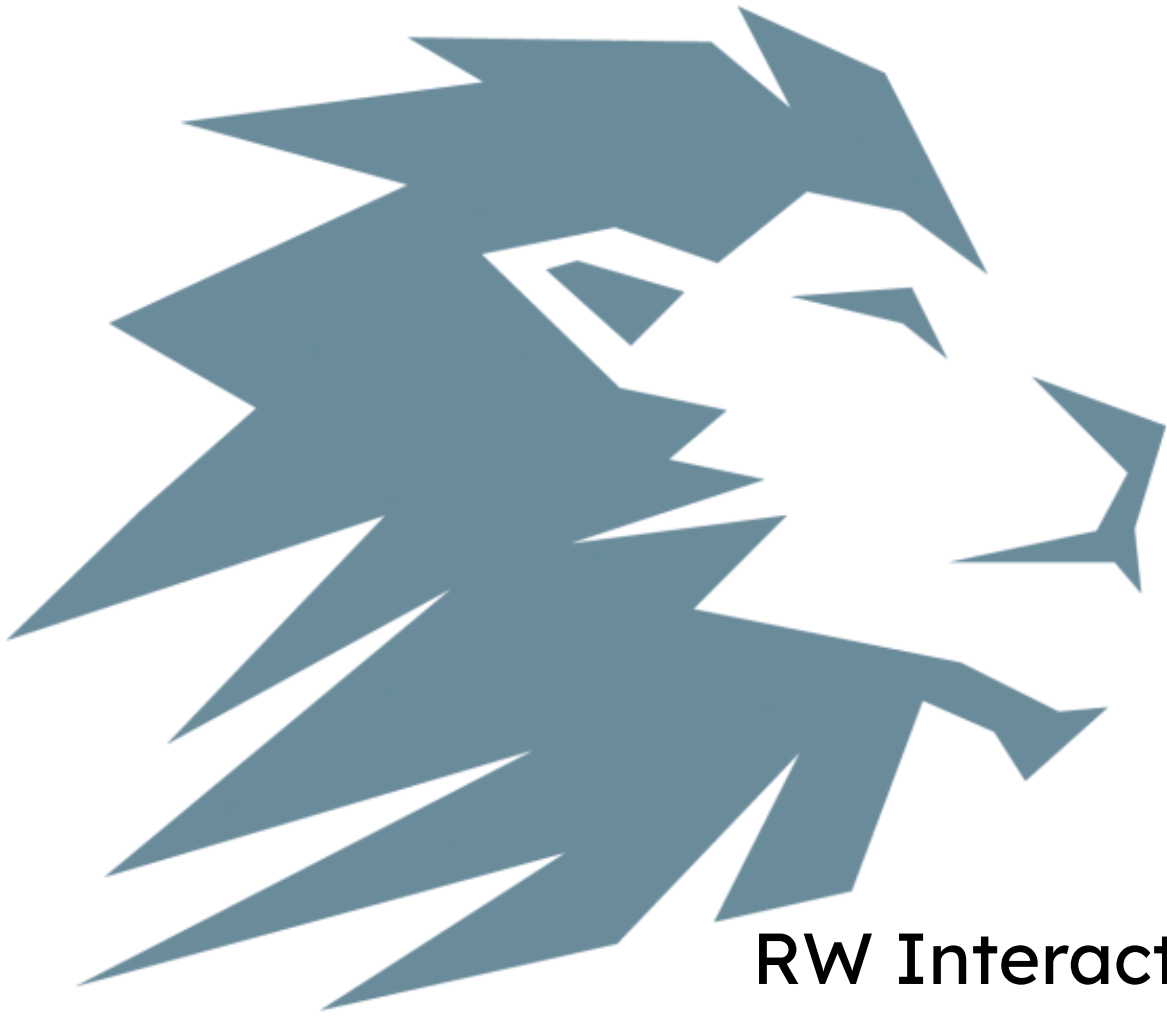




RW Interactive

Piquet - Deploy



Piquet — Deploy

1. A regra essencial

A branch **production** é a que faz deploy em produção.

O Jenkins constrói e publica automaticamente. Não há passo de aprovação nem rollback automático, e existe uma única máquina de produção.

A validação é feita **em desenvolvimento, antes do push**.

2. O que acontece no push

push → production (backend)



Jenkins (agent: Linux-2-8)

1. Cleanup & Checkout — clona backend@production
 - └─ clona payshop-sdk@production para ./payshop-sdk/
 - └─ ajusta o composer.json: "../payshop-sdk" → "./payshop-sdk"
2. Prepare Golden Image — imagem base cacheada por md5 do base.Dockerfile
3. Build & Push App Image — :<BUILD_NUMBER> e :latest → Docker Hub
4. Security Scan (Trivy) — relatório HTML (informativo)
5. Deploy (agent: master)
 - └─ ssh deployer@149.36.249.12
 - └─ escreve o .env (a partir do Jenkins)
 - └─ aponta o docker-compose.yaml à nova tag
 - └─ docker compose up -d
 - └─ docker image prune -f



Referência

Registry	Docker Hub — devopsrwinteractive
Imagem base	devopsrwinteractive/laravel83-infra-base:<md5 de infra/docker/base.Dockerfile>
Imagem da app	devopsrwinteractive/piquet-laravel-prod:<BUILD_NUMBER> (+ :latest)
Máquina	SRV-PIQU00-001 — 149.36.249.12
Utilizador de deploy	deployer
Diretoria	/home/deployer/project-laravel/
Compose	/home/deployer/project-laravel/docker-compose.yaml (.yaml)
Servidor web	nginx (host) → FrankenPHP (container laravel-prod)
Filas	Horizon, sob supervisord, no mesmo container

A tag da imagem é o número de build do Jenkins, não o commit. [:335](#) = build 335. Para identificar o código que está a correr, comparar o conteúdo do container com o git:

```
docker exec laravel-prod head -40 /var/www/html/routes/web.php
```

3. Testes

A suite corre **em desenvolvimento, antes do push** — o pipeline não a executa.



```
./vendor/bin/sail artisan test          # tudo
./vendor/bin/sail artisan test --testsuite=Unit
./vendor/bin/sail artisan test --filter=RateServiceTest
```

Suites em `phpunit.xml`: `tests/Unit` e `tests/Feature`.

Correr a suite antes de fazer push para `production`. Como o pipeline não a corre, é o único ponto onde uma regressão é apanhada.

A fase `Security Scan` corre o Trivy (`CRITICAL, HIGH`) e publica um relatório HTML no Jenkins. É **informativa** — não interrompe o deploy. Vale a pena consultá-la depois de um build.

4. Configuração de produção

O `.env` de produção está no Jenkins, como *Secret File* com o ID `piquetProdEnv`.

Cada deploy escreve-o na máquina, substituindo o que lá estiver.

Consequências práticas:

- Editar o `.env` na máquina não tem efeito duradouro — o próximo deploy substitui-o.
- Para alterar uma variável de produção: alterar a credencial `piquetProdEnv` no Jenkins e fazer um deploy.
- São ~100 variáveis. Não estão em nenhum repositório.
- Não existe `.env` dentro do container — a configuração entra pelo `docker-compose.yaml`.

Variáveis a conhecer

Variável	Produção	Nota
<code>APP_ENV</code>	<code>production</code>	
<code>APP_DEBUG</code>	<code>false</code>	Nunca true em produção — ativa ferramentas de



Variável	Produção	Nota
		desenvolvimento no backoffice, incluindo um atalho de autenticação.
APP_URL	<code>https://app.piquetapp.com</code>	Tem de corresponder ao host dos callbacks do Payshop; caso contrário as assinaturas falham.
MOCK_SMS	não definida	Não definir em produção. O serviço passa a devolver "Code sent" sem enviar SMS e sem devolver código — o login por telemóvel deixa de funcionar sem qualquer erro. O <code>.env.example</code> traz <code>MOCK_SMS=true</code> , pensado para desenvolvimento.
LOG_LEVEL	<code>debug</code>	
TRUSTED_PROXIES	não definida	
PAYSHOP_SDK_STRICT_SIGNATURE	não definida (<code>false</code>)	

Segredos vivem no Vault. Não circulam por chat nem por email.

5. Rollback (manual)

Não existe rollback automático. As imagens anteriores ficam no Docker Hub e, normalmente, também na máquina — o deploy só troca a tag no compose.

```
ssh deployer@149.36.249.12
```



```
cd project-laravel
```

```
# 1. Tags disponíveis localmente
```

```
docker images devopsrwinteractive/piquet-laravel-prod --format  
"{{.Tag}}\t{{.CreatedAt}}"
```

```
# 2. Guardar o estado atual antes de mexer
```

```
cp docker-compose.yaml docker-compose.yaml.bak-$(date +%Y%m%d-%H%M%S)
```

```
grep piquet-laravel-prod docker-compose.yaml          # anotar a tag atual
```

```
# 3. Apontar à tag anterior
```

```
sed -i 's|piquet-laravel-prod:.*|piquet-laravel-prod:<TAG_ANTERIOR>|g'  
docker-compose.yaml
```

```
# 4. Aplicar
```

```
docker compose up -d
```

```
# 5. Verificar (não confiar apenas no exit code)
```

```
docker ps --filter name=laravel-prod --format "{{.Image}}\t{{.Status}}"
```

```
curl -s -o /dev/null -w "%{http_code}\n" https://app.piquetapp.com/up      #  
200
```

Limites do rollback:

- O **.env** não faz rollback. Se a origem do problema foi uma alteração de configuração, voltar atrás na imagem não a reverte.



- **As migrações não fazem rollback.** Se o deploy correu migrações, a base de dados fica à frente do código.
 - **Não correr `docker system prune`** na máquina: remove as imagens anteriores e com elas a possibilidade de rollback rápido.
-

6. Backend e payshop-sdk

O Jenkins clona a branch `production` do `payshop-sdk` **no momento do build**:

```
dir('payshop-sdk') {  
  git branch: 'production', credentialsId: 'gitlab-rwi',  
  url: 'https://gitlab.rwinteractive.net/piquet/payshop-sdk.git'  
}
```

A versão do SDK que vai para produção é a que estiver na branch `production` quando o build corre — não está fixada a um commit.

Nota sobre o layout: em desenvolvimento o SDK é **irmão** do backend (`../payshop-sdk`); no build é uma **subdiretoria** (`./payshop-sdk`), e o Jenkins ajusta o `composer.json` para isso. Não alterar o `composer.json` para corresponder ao build — quebra o ambiente local.

7. Apps móveis

As apps **não passam pelo Jenkins** e são distribuídas via **EAS**.

O EAS constrói a diretoria de trabalho, não uma branch. Fazer merge não distribui nada; a distribuição é um passo explícito. Confirmar sempre em que repositório e diretoria se está.



App	Repositório	Branch	OTA
Cliente	<code>piquet/app-costumer</code>	<code>Production</code> (P maiúsculo)	Não
Vendor	<code>piquet/app-vendor</code>	<code>production</code> (minúsculas)	Sim (<code>app.config.ts:32-39</code> , canal <code>production</code>)

- **app-costumer**: cada correção precisa de **build nativo EAS + revisão de loja + reinstalação**. Contar com dias, não horas — planejar os releases em função disso.
- **app-vendor**: o `runtimeVersion` é a versão do `package.json`. **Subir a versão faz com que as instalações existentes deixem de receber OTA** até atualizarem pela loja. Ou seja: subir a versão e enviar um hotfix OTA são coisas para releases diferentes.

O repositório **app-vendor** tem também uma branch **Production** (maiúscula) que contém a app de cliente, por motivos históricos. Está a ser removida. A app de vendor é a `production` (minúsculas); a app de cliente é o repositório **app-costumer**.

8. Verificar um deploy

1. Que imagem está a correr

```
ssh devops@149.36.249.12 'docker ps --filter name=laravel-prod --format  
"{{.Image}}\t{{.Status}}"'
```

2. Responde

```
curl -s -o /dev/null -w "%{http_code}\n" https://app.piquetapp.com/up  
# 200  
curl -s -o /dev/null -w "%{http_code}\n"  
https://app.piquetapp.com/backoffice/login # 200
```

3. Erros recentes (o log está em UTC)



```
ssh devops@149.36.249.12 'docker exec laravel-prod sh -c "grep -E  
\"production\".(ERROR|CRITICAL)\" /var/www/html/storage/logs/laravel.log |  
tail -5\"'
```

Ausência de erros não é prova de saúde. Sem tráfego não há erros. Depois de um deploy que toque em pagamentos, executar um pagamento real e confirmar o resultado.

9. Logs

O quê	Onde
Aplicação	<code>docker exec laravel-prod → /var/www/html/storage/logs/laravel.log</code>
Chamadas de saída a terceiros (Payshop, InvoiceXpress)	<code>.../storage/logs/requests.log</code> — não é um log de acessos
Acessos HTTP (entrada)	nginx, no host: <code>/var/log/nginx/</code> (requer o grupo <code>adm</code>)
Filas / Horizon	<code>docker logs laravel-prod</code> (saída do worker; não contém acessos HTTP)

Fusos horários: os logs estão em **UTC**; a máquina está em **Europe/Lisbon** (WEST = UTC+1 no verão). Um relato de "às 06:50" corresponde a ~05:50 no log.

10. Em caso de problema em produção

1. Não reiniciar serviços como primeira tentativa.
2. Recolher evidência primeiro: que erro, a que horas (**UTC**), que utilizador, que pedido.
3. Se for necessário rollback, fazer backup do `docker-compose.yaml` antes.